GEMINI.md - Nisol Discovery Phase 1 (Antigravity Single Source of Truth)

0. Project Identity

Project: Nisol Discovery - Questionnaire Module (Phase 1)
Goal: Dead-simple, lightning-fast data capture tool for consultants. One question per page. "The stethoscope of Nisol Labs".
Version: 1.0 | Target: Google Antigravity

CRITICAL PRODUCT RULES - DO NOT VIOLATE:

1. NEVER perform INSERT/UPDATE/DELETE on `audit_maturity_scores` table. Reserved for Phase 2 Studio.

2. DO NOT build AI Report Generation, ROI Calculations, Maturity Scoring Algorithms, Opportunity Matrix.

3. Treat `raw_responses` JSONB as black box storage: `{"question_id": {"text": "...", "score": 1-5}}`

4. DO NOT create custom Next.js API routes. Use Supabase client directly.

5. Focus is UX speed: Page load <1.5s, Question nav <200ms, Auto-save every 5s debounce.

1. Tech Stack (LOCKED)

- Next.js 15 App Router, React 19, TypeScript 5.x, Tailwind CSS 3.x
- shadcn/ui (Radix: Button, Dialog, Input, Textarea, RadioGroup, Progress, Table, Card, Badge)
- Supabase: @supabase/supabase-js v2, PostgreSQL, Auth, RLS
- Form: React Hook Form (optional), State: Zustand or Context for Auth + current audit
- Hosting: Vercel, Env: NEXT_PUBLIC_SUPABASE_URL, NEXT_PUBLIC_SUPABASE_ANON_KEY

- Hosting/CI/CD: Vercel, automatic deploy from GitHub

2. Brand & UI System (LOCKED)

```
js
```

```
colors: {
  primary: "#0A1E3C", // Navy
  secondary: "#EBB44B", // Gold
  background: "#F8FAFC",
  card: "#FFFFFF",
  text: "#1E293B",
  muted: "#64748B",
  success: "#22C55E",
  warning: "#F59E0B",
  error: "#EF4444"
}
font: "Inter" Google Font
Headings: font-semibold text-2xl
Body: text-base
Tips: text-sm italic with bg-slate-100
```

Consultant Workbench Layout:
Header: [Logo] Nisol Discovery | [Progress: X/62] | [Profile] [Logout]
Sub-header: Capability: {section} (Section Y of 14) + Progress Bar (value = (currentIndex+1)/62*100)
Core: Large Question Text (text-base), Large Textarea for notes
Bottom row: MATURITY SCORE: [1][2][3][4][5] (radio, 1=Very Low, 5=Very High)
Right Sidebar Sticky: DISCUSSION GUIDE (tip_discussion) + TRIGGERED PATTERNS (pills from triggered_patterns array)
Footer: [◀ Previous] [Save & Continue] [Next ▶] -> On Q62, Next = "Complete Assessment" => sets status='data_collected'

3. Roles & RLS (ENFORCE STRICTLY)

Roles ENUM: user_role = 'super_admin' | 'admin' | 'consultant' | 'client'
Status ENUM: audit_status = 'draft' | 'data_collection' | 'data_collected' |

'in_analysis' | 'report_ready' | 'presented'

Permission Matrix:

- Super Admin/Consultant: Can see all tenants, all audits, Create Tenant, Create Audit, Write answers, Edit/Delete tenant/audit
- Client: Can ONLY see own tenant (where id = get_user_tenant()) and own tenant's audits, Read-Only questionnaire. Shows 🔒 Read-Only badge, all inputs disabled.
- Questions table: Anyone can read, only super_admin/admin can manage

Helper SQL functions MUST exist: get_user_role(), get_user_tenant(), is_internal_user()
RLS Policies: (Implement exactly as in SRS)

- tenants: Internal see all, Clients see own, Internal manage all
- audits: Internal see all, Clients read-only own, Internal create/update/delete
- questions: Anyone can read, Admins manage
- profiles: Users read own OR internal sees all, Users update own only

Trigger: on_auth_user_created -> INSERT into public.profiles (id, full_name, role='client')

4. Database Schema (EXACT - RUN THIS FIRST)

sql

```sql
CREATE EXTENSION IF NOT EXISTS "uuid-ossp";
CREATE TYPE user_role AS ENUM ('super_admin', 'admin', 'consultant',
CREATE TYPE audit_status AS ENUM ('draft', 'data_collection', 'data_c

CREATE TABLE public.tenants (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name TEXT NOT NULL, industry TEXT, employee_count INT, revenue_rang
  created_at TIMESTAMPTZ DEFAULT now(), updated_at TIMESTAMPTZ DEFAUL
);
```

```
CREATE TABLE public.profiles (
  id UUID PRIMARY KEY REFERENCES auth.users(id) ON DELETE CASCADE,
  tenant_id UUID REFERENCES public.tenants(id) ON DELETE SET NULL,
  full_name TEXT, role user_role NOT NULL DEFAULT 'client',
  created_at TIMESTAMPTZ DEFAULT now(), updated_at TIMESTAMPTZ DEFAUL
);
CREATE TABLE public.audits (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id UUID NOT NULL REFERENCES public.tenants(id) ON DELETE CAS
  title TEXT NOT NULL, conducted_by UUID REFERENCES public.profiles(i
  conducted_at TIMESTAMPTZ DEFAULT now(), status audit_status DEFAULT
  notes TEXT, raw_responses JSONB DEFAULT '{}'::jsonb,
  overall_maturity_score NUMERIC(3,2), -- NULL in Phase 1
  created_at TIMESTAMPTZ DEFAULT now(), updated_at TIMESTAMPTZ DEFAUL
);
CREATE TABLE public.questions (
  id SERIAL PRIMARY KEY, section TEXT NOT NULL, order_index INT NOT N
  question_text TEXT NOT NULL, tip_discussion TEXT, triggered_pattern
  created_at TIMESTAMPTZ DEFAULT now()
);
CREATE TABLE public.audit_maturity_scores (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  audit_id UUID NOT NULL REFERENCES public.audits(id) ON DELETE CASCA
  pillar_name TEXT NOT NULL, maturity_score NUMERIC(3,2) NOT NULL,
  recommendations TEXT, created_at TIMESTAMPTZ DEFAULT now()
);
CREATE INDEX idx_audits_tenant_id ON public.audits(tenant_id);
CREATE INDEX idx_audits_conducted_by ON public.audits(conducted_by);
CREATE INDEX idx_audits_status ON public.audits(status);
CREATE INDEX idx_profiles_tenant_id ON public.profiles(tenant_id);
-- Add RLS, Functions, Policies, Trigger as defined in SRS Section 6
```

5. Supabase Integration Pattern (MANDATORY)

Use direct client, NO API routes:

- Fetch Questions: supabase.from('questions').select('*').order('order_index')
- Fetch Tenants: supabase.from('tenants').select('*').order('name')
- Create Tenant: supabase.from('tenants').insert({name, industry})
- Fetch Audits: supabase.from('audits').select('*, tenants(name)').order('created_at', {ascending:false})

- Create Audit: supabase.from('audits').insert({tenant_id, title, conducted_by, status: 'draft'})
- Fetch Single Audit: supabase.from('audits').select('*, tenants()').eq('id', auditId).single()
- Update Audit: supabase.from('audits').update({raw_responses: updatedJson, status: 'data_collection'}).eq('id', auditId)
- Complete Audit: supabase.from('audits').update({status: 'data_collected'}).eq('id', auditId)

6. Pages to Build (IN ORDER FOR ANTIGRAVITY)

6.1 /login - Public

Email, Password, Login button, Forgot Password. Logic: supabase.auth.signInWithPassword() -> /dashboard. Use shadcn Card.

6.2 /dashboard - Authenticated

Internal: Table Title | Client (Tenant) | Status Badge | Date | Actions + [New Assessment] button.
Client: Filtered where tenant_id = get_user_tenant(). Table same but read-only. Use @/components/ui/table, Badge variants: draft=secondary, data_collection=warning, data_collected=success.

6.3 /clients - Super Admin, Consultant only

Search bar, Add Client button (Dialog), Table: Name, Industry, Employees, Actions (Edit, Delete). Confirm delete/create.

6.4 /audits/new and /audits/[id]

Internal only. Fields: Title (Input), Tenant (Select dropdown from tenants), Consultant (pre-filled current user full_name). On submit -> create audit -> redirect to /audits/[id]/questionnaire

6.5 /audits/[id]/questionnaire - THE CORE (80% effort here)

State:

- const { data: audit } = fetch audit
- const { data: questions } = fetch all questions order_index
- const [currentIndex, setCurrentIndex] = useState(audit.raw_responses ? last answered index : 0) -> persist in localStorage for state recovery
- const currentQuestion = questions[currentIndex]
- const currentAnswer = audit.raw_responses[currentQuestion.id] || {text:'', score:null}

UI Components to Create:

- @/components/layout/Header (Logo, Progress X/62, Profile, Logout)
- @/components/layout/Sidebar (user, role)
- @/components/questionnaire/QuestionCard (question_text, section)
- @/components/questionnaire/AnswerTextArea (large textarea, bind to text)
- @/components/questionnaire/ScoreSelector (RadioGroup 1-5)
- @/components/questionnaire/DiscussionPanel (tip_discussion, triggered_patterns as Badge pills)
- @/components/ui/progress (for top bar)

Save Logic:

1. On TextArea change + Score change -> debounce 5s -> call updateAudit (merge JSONB: {...audit.raw_responses, [questionId]: {text, score}})

2. On Previous/Next click -> immediate save -> setCurrentIndex +/- 1 -> if restoring tab, use saved index

3. On Q62 Next -> Button text "Complete Assessment" -> update status='data_collected' -> redirect /dashboard

4. Disable inputs if role === 'client' -> show 🔒 Read-Only badge top right

6.6 /profile

Editable Full Name, Read-only Role, Tenant (for client). Change Password via supabase.auth.updateUser()

7. Component Library Paths (ENFORCE)

@/components/ui/button, input, textarea, radio-group, progress, table, card, badge, dialog
@/components/layout/Header, Sidebar
@/components/questionnaire/QuestionCard, DiscussionPanel

Props examples:
QuestionCard: { question, answer, onSave }
DiscussionPanel: { tips: tip_discussion, patterns: triggered_patterns }
Sidebar: { user, role }

8. Questionnaire Flow (SEQUENCE)

1. Consultant login -> Dashboard -> New Assessment -> Select/Add Client -> Enter Title -> Start -> Redirect /audits/{id}/questionnaire

2. Load Q1/62 -> Discuss with client -> Enter notes -> Select Score 1-5 -> Next -> save to raw_responses JSONB -> increment index

3. Repeat to Q62 -> Next becomes Complete -> Save -> status='data_collected' -> Dashboard

4. Client login -> sees audit Read-Only mode

9. What NOT to Build (Antigravity must skip)

- No audit_maturity_scores writes
- No charts, graphs, analytics
- No AI generation, no ROI, no maturity calculation
- No email notifications
- No mobile native, responsive web only for 14" laptop primary
- No advanced filtering beyond tenant name search

10. Antigravity Execution Instructions

1. Step 1: `npx create-next-app@latest nisol-discovery --typescript --tailwind --eslint --app --src-dir --import-alias "@/*"` + install shadcn, supabase

2. Step 2: Create .env.local with Supabase keys, create lib/supabase/client.ts and server.ts

3. Step 3: Run full SQL schema above in Supabase SQL editor (provide sql file for user to run)

4. Step 4: Build Auth (middleware.ts for protected routes except /login)

5. Step 5: Build Dashboard, Clients, New Audit pages

6. Step 6: Build Questionnaire Core with auto-save, progress, read-only logic, state recovery

7. Step 7: Seed 62 questions - create placeholder seed file `supabase/seed_questions.sql` with 3 sample questions, leave TODO for user to paste remaining 61

8. Step 8: Test RLS with 3 users: super_admin, consultant, client

Performance: Use React.memo for QuestionCard, useCallback for save, keep navigation <200ms by prefetching next question.

When you start, first read this file and the attached SRS docx. Build strictly to this spec.